

Physicalized Model Weights Final Synthesis

2026-05-13

Contents

Physicalized Model Weights Final Synthesis	1
Abstract	1
Introduction	1
1. Research Question, Definitions, and Null Hypothesis	2
2. Target Selection and the Hybrid Safety/Filter Architecture	3
3. Prototype and First Evidence-Labeled Synthesis	4
4. Calibration, Workload Replay, and Phase 2 Downgrade	5
5. Production Measurement Contract and Trace Validation	7
6. Reopen Pathway: Thresholds, Ingestion, Evidence Packs, and Uncertainty	8
7. Lifecycle State Machine and Canonical Phase 4 Reopen Condition	10
8. Robustness, Closure, and Archive	10
9. Post-Closure Checks and Public-Baseline Refresh	12
10. Trigger-Gated Endpoint and Residual Work	13
Conclusions	14
Implementation Details	14
References	16

Physicalized Model Weights Final Synthesis

Abstract

This report synthesizes the investigation into whether useful parts of neural-network inference can be moved from programmable execution into fixed, semi-fixed, or physically encoded hardware. The work rejected broad fixed frontier-model physicalization under the current evidence. It retained a narrower safety/filter classifier architecture as a bounded architecture, verification, failure-mode, and evidence-gating study, but calibrated modeling and stronger programmable baselines falsified the current performance or economic superiority claim. A future reopen requires lifecycle-valid measured production, shadow-production, or canary evidence that compares the measured hybrid total against the measured best programmable baseline under identical accounting, crosses the quantitative threshold, and leaves the upper confidence bound on $H - B$ below zero. The final audit headline supplied for this report is `promise_check=unknown`; it also reports complete figure coverage, with 33 figures present and 33 figures in the ledger, and no missing or orphan figures. The wall-cap flag is false.

Introduction

The original directive asked for an autonomous long-exposure investigation of physicalized model weights: cases where part of model inference is encoded into hardware structure, memory placement, topology, or fixed compute behavior rather than executed only as mutable software on a programmable processor. The central question was not whether a fixed hardware implementation can be made faster on a narrow arithmetic kernel. The question was whether such a design remains advantageous after update cadence, fallback behavior,

audit requirements, feature extraction, integration overhead, utilization, yield, and stronger programmable baselines are included.

This report is a synthesis of the completed research record. It presents the definitions, target-selection rationale, prototype result, calibrated downgrade, production-evidence contract, reopen pathway, lifecycle rule, closure state, public-baseline refresh, and endpoint conditions. It reports the accepted results from the completed research record and final audit inputs; it does not re-audit those results.

1. Research Question, Definitions, and Null Hypothesis

The research question was whether useful parts of neural-network inference can be moved from ordinary programmable execution into fixed, semi-fixed, or physically encoded hardware. The work did not assume that an entire frontier language model should be permanently burned into a chip. From the first report, the intended question was narrower: which inference components are stable, frequently reused, energy-relevant, and tolerant enough that physicalization could beat optimized programmable execution after update cadence, utilization, yield, repair, and integration costs are counted.

In this research record, “physicalized model weights” means encoding part of an inference workload into hardware structure, storage, topology, data movement, or compute behavior. The initial taxonomy included fixed digital logic, ROM-coded weights, SRAM or eDRAM-resident weights, FPGA and embedded-FPGA overlays, RISC-V-attached accelerators, chiplet or cartridge-style weight modules, analog in-memory arrays, photonic or mixed-signal paths, and hybrid systems that keep some work fixed while leaving control programmable. This definition made physicalization a spectrum rather than one device proposal.

The central discipline was the null hypothesis: optimized software, runtime, compiler, scheduler, memory-management, batching, routing, quantization, caching, and programmable-accelerator improvements may capture much of the available gain without fixing weights in hardware. The initial plan therefore required physicalization to be compared against strong non-physicalized baselines, not against an artificially weak CPU or GPU implementation.

The first analytical mechanism was a break-even inequality:

$$N * (C_{\text{prog}} - C_{\text{phys}}) > C_{\text{fixed}} + C_{\text{update}} + C_{\text{yield}} + C_{\text{integration}}$$

Here, N is the number of served requests or tokens before a material update. C_{prog} is the best programmable per-request cost after software and runtime optimization. C_{phys} is the per-request cost of the physicalized path after conversion, utilization, yield or repair, and fallback penalties. The right-hand side represents fixed substrate cost, update cost, yield or repair cost, and integration cost.

The inequality set the interpretation of the whole run. If request volume is zero, fixed physicalization cannot amortize nonzero fixed cost. If programmable execution is already cheaper than physicalized execution, no finite positive request count produces a win. If update intervals become too short, fixed weights can become stranded before they pay back their substrate cost. If software/runtime optimization cuts memory movement substantially, the request volume required for physicalization rises. If analog conversion, calibration, yield, or fallback penalties are high, analog array-level savings can disappear even when raw array read energy is low.

The break-even modeling phase turned this framing into executable artifacts, including a normalized Python sweep, a Wolfram symbolic derivation, and corresponding tests and outputs.

The Python model compared six strategies: unoptimized programmable inference, software-optimized inference, programmable acceleration, fixed digital weights, analog in-memory compute, and a hybrid physicalized submodel. It emitted 3,360 sweep rows. In the reported default sweep, fixed digital weights won 208 rows, programmable acceleration won 192 rows, and software-optimized inference won 160 rows. Analog in-memory and hybrid physicalized submodels did not win under the default or bad-penalty assumptions reported in the model summary.

The Wolfram script reduced the break-even threshold to $N_{\text{break_even}} = \text{fixedCost} / (\text{cProg} - \text{cPhys})$ and checked the special cases above. That result was not treated as proof of a hardware win. It was used as a filter: physicalization was only worth pursuing where reuse volume, update stability, and per-request savings could plausibly offset fixed and integration costs.

The first important decision followed from this foundation. The research would not proceed by designing a broad fixed-weight language-model chip. It would first find a bounded target whose stability, reuse, and failure behavior made it a defensible physicalization candidate.

2. Target Selection and the Hybrid Safety/Filter Architecture

The target-ranking work rejected full frontier-model permanence and selected a fixed or semi-fixed safety/filter classifier submodel as the strongest physicalization target. This was a narrowing decision, not a proof that the target would ultimately win.

The target-selection phase scored candidate and anti-target components by update stability, reuse volume, approximation tolerance, integration complexity, energy upside against baselines, resistance to software-baseline improvements, and evidence quality. At 35% software/runtime memory-movement savings, the top ranked candidates were fixed safety/filter classifier submodels, small always-on wake-word or router models, repeated retrieval or reranking feature transforms, and mixture-of-experts router or dispatch logic. The lowest-scored anti-targets included full frontier dense weights permanently burned into fixed logic, frequently updated tenant-specific adapters, high-churn vocabulary or logit-head variants, dynamic attention over live context as fixed logic, and training or optimizer state.

The selected target was a safety/filter classifier because it had the right shape for a bounded test. It could be small, isolated, repeatedly invoked, conservatively versioned, and routed around when confidence, health, policy version, or audit requirements failed. It also avoided the main anti-target properties: broad model churn, live context dependence, tenant-specific updates, and direct ownership of user-facing generation.

The architecture phase converted that target into a hybrid architecture. The physicalized classifier sits before the main serving stack and receives bounded request features from the host runtime. It emits a classifier decision and confidence, but it does not own tokenization, prompt parsing, policy authoring, dynamic attention state, or final routing. The host remains responsible for policy versioning, health checks, drift checks, audit logging, rollback, and fallback.

The architecture used a host-controlled, memory-mapped accelerator boundary with a RISC-V-compatible integration path. RISC-V was used as an open control-plane anchor because it is a free and open instruction set architecture with public technical specifications [1], [2]. The reports did not treat RISC-V as automatically optimal; they used it as a credible open substrate for a small, inspectable accelerator interface.

The register map made the boundary explicit. It covered device identity, ABI version, status, control, request ID, feature address and length, policy slot, required and active policy versions, threshold, confidence, decision, fallback reason, audit ring state, and rollback slot.

This register map mattered because the fixed classifier was intentionally not trusted as a complete policy system. It was a fast path inside a larger programmable safety and control envelope.

The fallback-policy simulator made the architecture executable. It tested 11 policy cases and produced three route classes:

Route	Count
physicalized_fast_path	2
programmable_fallback	7
fail_safe	2

Only healthy, high-confidence, version-valid cases reached the physicalized fast path. Stale policy, low confidence, zero confidence, failed health, drift alarm, host-forced fallback, and audit logging failure routed away from the physicalized output. Invalid classifier output with unavailable fallback entered fail-safe.

This result defined the first stable architecture claim. The credible system was not a standalone “weight chip.” It was a programmable serving path that could optionally route a narrow, stable classifier through fixed or semi-fixed hardware. The physicalized component was deliberately kept subordinate to programmable update, audit, fallback, rollback, and final-routing logic.

3. Prototype and First Evidence-Labeled Synthesis

The prototype phase turned the validated architecture into a small implementation and then consolidated the first research arc. The prototype did not establish a production hardware win. It showed that the proposed boundary could be implemented as an inspectable fixed classifier with conservative route and fallback behavior.

The prototype classifier used 8 signed int8 features, fixed signed int8 weights, a bias of -10, and a threshold of 64. Its fixed weights were:

```
[12, -7, 5, 9, -11, 4, 6, -3]
```

The classifier returned `block` when the score was greater than or equal to the threshold and `allow` otherwise. Confidence was defined as distance from the threshold. The route decision then checked whether the fixed classifier output could be used at all. Low confidence, stale policy version, failed health, drift alarm, host-forced fallback, audit failure, classifier unavailability, or invalid output routed away from the physicalized fast path.

Across 16 prototype cases, the route distribution was:

Route	Count	Fraction
physicalized_fast_path	6	0.375
programmable_fallback	8	0.500
fail_safe	2	0.125

The local baseline comparison used normalized cost units and included feature extraction, register/control, audit, fallback dispatch, and fail-safe handling. It reported 400.0 modeled cost units for optimized software, 368.0 for a programmable accelerator, and 162.0 for the

hybrid physicalized path over the 16 diagnostic cases. The reports interpreted this as architecture support only. Because the units were normalized and the cases were diagnostic rather than production traces, the result did not establish measured economic or performance superiority.

The hardware description language work was similarly bounded. The HDL core was pure combinational logic: fixed localparam weights, signed dot product, threshold comparison, and margin/confidence outputs. There was no clock, reset, memory, handshake timing, mutable policy logic, or sequential state. That narrow scope allowed the verification contract to stay modest.

Prototype verification confirmed Python prototype generation, Yosys evaluation, Verilator lint, Yosys synthesis, Graphviz rendering, and stdlib tests. Compiled Verilator simulation did not run because the environment lacked `make`. The evidence contract was then amended rather than treating compiled simulation as passed. The closure accepted Verilator lint, Yosys evaluation, Yosys synthesis, Graphviz artifacts, Python golden-vector agreement, and artifact hash freshness as sufficient for this specific combinational core. The closure JSON recorded `closure_status = validated`, `compiled_simulation_status = blocked_make_unavailable`, `yosys_eval_matches_python = true`, `verilator_lint_passed = true`, `yosys_synthesis_passed = true`, `graphviz_artifacts_present = true`, and `structural_artifacts_fresh = true`.

The prototype evidence contract also defined reopen conditions for the prototype milestone. M-PROTO-1 should reopen if compiled Verilator later runs and disagrees, Python and Yosys rows diverge, the HDL hash changes without regenerated evidence, or the HDL grows to include sequential state, memories, handshake timing, or mutable policy logic. This preserved the scope of the validation: the accepted evidence applies to the tiny combinational classifier, not to a larger accelerator.

The first synthesis for the initial arc consolidated the evidence into labeled categories: `sourced`, `modeled`, `simulated`, `synthesized`, `inferred`, and `speculative`. After a manifest fix, the evidence map contained 25 artifacts and separated architecture support from production or economic evidence.

The first-arc conclusion was narrow and conditional. The completed work did not support permanently fixing full frontier large-language-model weights in hardware. It supported continued study of a stable, bounded, high-reuse safety/filter submodel only when placed behind programmable fallback, signed update, audit, health, drift, and rollback controls. The prototype showed that such a boundary could be made small and inspectable. It also reinforced the null hypothesis: most practical system complexity remained in feature extraction, runtime control, policy management, audit, fallback, and programmable serving infrastructure.

4. Calibration, Workload Replay, and Phase 2 Downgrade

The Phase 2 result was a downgrade of the remaining safety/filter physicalization claim. Calibration first weakened the claim, workload replay then narrowed it to one preserved synthetic workload, and a stronger programmable accelerator baseline erased that last modeled win. After this point, the research no longer supported the statement that the hybrid physicalized safety/filter path was a performance or economic winner over strong programmable baselines.

The calibration step replaced the earlier normalized model with explicit-unit companion modeling. The reports used public energy-scale and accelerator framing sources as context: Horowitz operation and memory-access energy material [7], [8], NVIDIA H100 public accelerator information [9], and MLPerf Inference documentation for system-level benchmark framing [10]. The calibration work also added local host/Python proxy measurements for small int8

dot products, dispatch, combined dot-plus-dispatch, and audit logging. Those measurements were treated as local overhead proxies, not silicon energy evidence.

The calibrated model evaluated 6,300 scenarios. The reported winners were:

Winner	Scenario count
programmable accelerator	4,948
optimized software	900
hybrid physicalized safety/filter	452

The hybrid safety/filter winner share fell to 0.0717, with the result classified as `preserved_but_weakened`. The top uncertainty drivers were fallback frequency, utilization, requests per day, audit/control scale, and update interval. This shifted attention away from the fixed dot-product core and toward the surrounding serving system.

The workload replay made that shift concrete. Ten deterministic synthetic workload regimes distinguished raw request volume from effective fast-path volume. Effective fast-path volume is the request share that can actually use the fixed classifier after fallback, near-threshold uncertainty, stale policy windows, drift, audit failures, fallback outages, and utilization penalties are counted. Only one regime, `high_volume_stable_moderation`, preserved the physicalized safety/filter claim. Three were weakened, two were speculative, and four were falsified. The preserved case had high traffic, low fallback pressure, monthly-or-slower update cadence, and bounded audit/control overhead.

The decisive test was the stronger-baseline replay. The same workload rows were compared across optimized software/runtime, programmable accelerator, and hybrid physicalized safety/filter alternatives under identical traffic, feature extraction, fallback, audit, update, and utilization accounting. The result was:

Winner	Scenario count
programmable accelerator	9
optimized software/runtime	1
hybrid physicalized safety/filter	0

For the previously preserved `high_volume_stable_moderation` case, the programmable accelerator cost proxy was 8,178,811,874.414918 pJ-equivalent/day, while the hybrid physicalized safety/filter cost proxy was 9,650,260,720.03919 pJ-equivalent/day. The hybrid margin against the best programmable baseline was therefore -1,471,448,845.624272 pJ-equivalent/day. A negative margin means the best programmable baseline was already cheaper under the model.

The reports later use H for the measured or modeled hybrid physicalized total and B for the measured or modeled best programmable baseline total. Phase 2 established that, for the current modeled workload set, $H - B$ was not below zero for any scenario. The safety/filter hardware remained useful as an architecture and failure-mode study, but not as a demonstrated winner.

The Phase 2 synthesis converted this downgrade into the current claim set. The Phase 2 claim matrix marked the physicalized safety/filter performance/economic winner claim as `falsified`. It preserved the architecture and failure-mode study as useful, and it marked earlier target-ranking superiority language as superseded because target suitability alone no longer justified performance-superiority language after stronger-baseline replay.

5. Production Measurement Contract and Trace Validation

The next decision was that local timings and synthetic traces could not reopen the downgraded claim. The production-evidence phase defined a measurement contract and then encoded it in a trace validator. The result was a hard separation between tooling evidence and production evidence.

The local proxy benchmark measured six components over the ten validated workload scenarios:

Component	Median proxy latency
append-only audit write	20538.441 ns/request
audit serialization	17367.176 ns/request
feature extraction	10437.758 ns/request
optimized software classifier	8763.939 ns/request
route/fallback decision	5667.545 ns/request
fixed classifier	5160.697 ns/request

The summary reported that control overhead dominated the fixed classifier. This result supported the system-level interpretation of Phase 2: the fixed arithmetic block was not the main unresolved cost. Feature extraction, audit serialization, audit writes, fallback routing, baseline execution, and accelerator integration had to be measured under the same workload before any hardware-superiority claim could be reconsidered.

The measurement gap matrix then separated what had been locally proxied from what remained production-required. Local proxies covered feature extraction latency, audit serialization/logging latency, fallback dispatch timing, and optimized software classifier timing. Production-required or unmeasured quantities included feature extraction energy, audit storage cost, optimized software energy, programmable accelerator latency, programmable accelerator energy, accelerator utilization and batching, and durable hybrid margin.

The production trace schema made those requirements machine-checkable. A trace could only become a `valid_reopen_candidate` if it had nonzero request volume, nonzero accepted physicalized fast-path requests, measured accelerator energy, measured hybrid energy, required software and accelerator baseline latency fields, audit fields, passing health and drift gates for accepted fast-path rows, and a consistent policy window.

The validator also closed several loopholes. It rejected incomplete telemetry, proxy-only energy, privacy-risk columns, missing baselines, inconsistent policy windows, zero-volume controls, all-fallback controls, and failed fast-path guardrails. After an auditor fix, accepted fast-path credit also required `fallback_taken=false`, `audit_logged=true`, `health_gate_passed=true`, and `drift_gate_passed=true`.

The synthetic valid fixture remained intentionally non-reopening. It had nonzero request volume and accepted fast-path rows, but it was blocked because its hybrid energy was proxy evidence and its environment was synthetic. The invalid fixture was rejected for privacy-risk content, missing accelerator baseline latency, negative feature-extraction latency, and inconsistent policy versions.

This measurement contract changed the meaning of future work. The open question was no longer whether another local benchmark or synthetic workload could make the hybrid path look favorable. The required evidence became measured feature extraction, audit cost, fallback behavior, update cadence, utilization, software baseline latency and energy, pro-

grammable accelerator latency and energy, and hybrid total cost over the same requests and policy window.

6. Reopen Pathway: Thresholds, Ingestion, Evidence Packs, and Uncertainty

The reopen-pathway work made the future evidence path executable without changing the scientific conclusion. No current artifact reopened the Phase 2 downgrade. The contribution was a complete gate chain for future evidence: quantitative threshold, admissible ingestion path, end-to-end pipeline status, evidence package replay, acquisition readiness, operator dry run, intake rehearsal, and uncertainty-aware margin.

The quantitative threshold model stated the core reopen inequality:

```
measured_hybrid_total < measured_best_programmable_baseline
```

The comparison must use identical accepted-volume, fallback, audit, update, utilization, latency, and energy accounting. For the `high_volume_stable_moderation` scenario, the current gap was the same Phase 2 margin: the hybrid path would need a 1,471,448,845.624272 pJ-equivalent/day reduction, or the programmable baseline would need an equivalent degradation, just to tie. The threshold summary reported `current_hybrid_wins = 0`, eight finite-threshold scenarios, one unreopenable zero-volume scenario, and one unreopenable all-fallback scenario.

The ingestion-path evaluator then ranked possible evidence sources. It classified `shadow_production_dual_run` and `canary_ab_dual_instrumented` as `reopen_candidate_path` designs. `offline_replay_redacted_features` was only `threshold_evaluable_if_measured`. Synthetic fixtures, sampled production logs without baselines, vendor-only benchmarks, and simulated scaled workloads were valid or diagnostic in limited ways but insufficient to reopen the claim. Privacy-risk raw logs were inadmissible.

The end-to-end pipeline combined trace validation, ingestion admissibility, provenance checks, measured-status checks, and threshold comparison. Its final statuses were `invalid_trace`, `valid_but_insufficient`, `threshold_evaluable_not_crossed`, `synthetic_counterfactual_crossed`, and `actual_reopen_candidate`. The generated fixtures covered the non-reopening branches, and the summary reported `actual_reopen_candidate_count = 0`.

The evidence-pack layer bound the required inputs into replayable manifests. A package had to preserve the trace file hash, schema compatibility, privacy attestation, provenance attestation, ingestion path, evidence source type, measurement status, threshold scenario mapping, and downstream pipeline result. The replay summary covered five privacy-safe manifest fixtures: three valid packages, two invalid packages, and zero actual reopen candidates. An auditor fix made unknown threshold scenarios hard package blockers before downstream evaluation.

The Phase 3 synthesis consolidated the gate chain across measurement requirements, trace validation, thresholds, ingestion admissibility, pipeline status, and evidence-pack replay. It reported 14 Phase 3 claims, `current_artifacts_reopen = false`, `actual_reopen_candidates = 0`, and `ingestion_actual_reopened_count = 0`. It also named the blocked evidence classes: synthetic, proxy/local, vendor-only, privacy-risk, stale-hash, unknown-threshold, and non-crossing measured packages.

The acquisition-readiness layer screened proposed collection designs before data collection. It evaluated 20 criteria across ten proposed designs. Two designs, `shadow_dual_run_full_`

instrumentation and canary_ab_full_instrumentation, were ready_to_collect_candidate; one was repairable before collection; five were inadmissible; and two were diagnostic-only. The ready designs were not evidence. They were admissible plans for collecting future evidence.

The operator dry-run and intake rehearsal layers then made that future path usable. The dry-run checker evaluated package templates for manifest completeness, trace header privacy, required columns, threshold scenario allow-list membership, hash consistency, source/measurement consistency, and attestation replacement. Two complete templates were ready_for_collection_not_evidence, and ten cases were blocked for schema, template, privacy, provenance, integrity, or threshold-mapping problems. The intake rehearsal filled synthetic-safe package-local traces, computed SHA-256 hashes, wrote replay-compatible manifests, checked handoff preservation, and delegated preserved packages to the replay evaluator. Three synthetic-safe packages passed intake, six mutation cases were blocked before replay, and zero packages became actual reopen candidates.

The final reopen-pathway addition was uncertainty. A future measured package must not merely show a favorable point estimate. It must show a statistically durable margin. The protocol defined:

$$D = H - B$$

where H is the measured hybrid total and B is the measured best programmable baseline total under the same workload and accounting. The uncertainty-aware rule is:

$$UCB_{\alpha}(D) = \Delta_{\text{mean}} + z_{\alpha}\sigma_{\Delta} < 0$$

with:

$$\Delta_{\text{mean}} = H_{\text{mean}} - B_{\text{mean}}$$

and:

$$\sigma_{\Delta} = \sqrt{\sigma_H^2 + \sigma_B^2 - 2\rho\sigma_H\sigma_B + \sigma_{\text{workload mix}}^2 + \sigma_{\text{meter}}^2}$$

This rule is necessary but not sufficient. It is conjoined with all previous gates: valid package, hash match, schema compatibility, known threshold scenario, valid trace, admissible ingestion path, measured terms, production/shadow/canary source, provenance attestation, privacy attestation, threshold crossing, nonzero request volume, and nonzero accepted fast-path volume. The uncertainty summary evaluated 11 synthetic-safe cases and reported zero actual reopen candidates.

The resulting reopen condition is precise. A future challenge to the Phase 2 downgrade must be a measured, privacy-safe, provenance-attested production, shadow-production, or canary package; it must preserve hashes and schema compatibility; it must map to a known threshold scenario; it must pass trace validation and ingestion admissibility; it must include nonzero request and accepted fast-path volume; it must cross the quantitative threshold against the best programmable baseline under identical accounting; and its upper confidence bound on $H - B$ must be below zero. Current artifacts are templates, proxies, synthetic fixtures, rehearsals, and decision protocols. They do not satisfy that condition.

7. Lifecycle State Machine and Canonical Phase 4 Reopen Condition

The Phase 4 work converted the accumulated reopen gates into one lifecycle state machine and one canonical reopen rule. The conclusion did not change: no current artifact was actual measured production, shadow, or canary evidence, and no current artifact reopened the Phase 2 downgrade.

The lifecycle state machine classified candidate evidence packages through the existing gates: acquisition readiness, operator dry run, intake rehearsal, evidence-pack replay, threshold evaluation, and uncertainty margin. A lifecycle state machine is a deterministic classifier. Each package scenario receives a terminal state, an owning gate, and a rationale. Terminal states included `collection_ready_not_evidence`, `dryrun_ready_not_evidence`, `intake_rehearsed_not_evidence`, `replay_blocked`, `threshold_crossed_nonactual`, `uncertainty_inconclusive`, `statistically_durable_nonactual`, and `actual_reopen_candidate`.

The lifecycle summary reported 15 cases, zero current actual reopen candidates, `current_artifacts_reopen = false`, one hypothetical actual-candidate control, and zero status mismatches. The single `actual_reopen_candidate` terminal state belonged only to a hypothetical measured durable positive-control row. It showed that the state machine could represent future valid evidence; it was not counted as current evidence.

The Phase 4 synthesis then canonicalized the future reopen condition:

```
valid_package && hash_match && schema_compatible && known_threshold_scenario &&
valid_trace && admissible_ingestion_path && measured_terms &&
production_or_shadow_or_canary_source && provenance_attestation &&
privacy_attestation && nonzero_request_volume &&
nonzero_accepted_fast_path_volume && measured_best_programmable_baseline &&
threshold_crossed &&  $UCB_{\alpha}(H - B) < 0$  &&
lifecycle_terminal_state=actual_reopen_candidate
```

Here, H is the measured hybrid total and B is the measured best programmable baseline total under the same workload accounting. The expression $UCB_{\alpha}(H - B) < 0$ means the upper confidence bound on the hybrid-minus-baseline margin must remain below zero. A favorable point estimate is not enough.

The Phase 4 summary reported 10 claim rows, zero current actual reopen candidates, `current_artifacts_reopen = false`, and one hypothetical actual-candidate control. An auditability fix ensured every claim-support artifact appeared in the Phase 4 manifest. After that fix, the Phase 4 synthesis made the reopen condition complete and traceable without adding a new route around the earlier evidence gates.

8. Robustness, Closure, and Archive

The robustness, closure, and archive work converted the negative result into a governed endpoint. It did not soften the conclusion. Across these stages, the reported endpoint counters stayed fixed: zero current physicalized superiority claims, zero actual reopen candidates, zero new reopen gates, and `current_artifacts_reopen = false`.

The target robustness stress test evaluated eight target classes:

Target class
<code>safety_filter</code>
<code>embedding_lookup_or_static_table</code>
<code>fixed_feature_extractor</code>

Target class
small_keyword_or_policy_classifier
decoder_dense_weights
attention_kv_or_dynamic_context
tenant_adapter_or_lora
training_optimizer_state

The stress test covered calibrated, favorable-plausible, extreme-counterfactual, and special-control regimes. Its final summary reported 28 cases, zero calibrated physicalized wins, four favorable-plausible physicalized wins, zero plausible anti-target wins, eight extreme counterfactual wins, zero current superiority claims, and zero status mismatches. The favorable-plausible wins were limited to candidate-like stable targets under simultaneous favorable movement in volume, utilization, update cadence, overhead, and physicalized per-request savings. Extreme wins were labeled counterfactual, not current evidence.

The deferral watchlist then separated claims, future triggers, and insufficient substitutes. It recorded five current claim dispositions: broad fixed frontier-model physicalization rejected under current evidence; safety/filter performance or economic superiority falsified under the stronger programmable baseline; hybrid architecture and prototype retained as architecture, failure-mode, and evidence-scaffold work; the Phase 4 reopen pathway complete but inactive; and no calibrated current superiority claim for non-safety target classes.

The watchlist also made future triggers explicit. Measured shadow/canary packages, measured production packages, and new stable high-volume target evidence remained inactive unless they satisfied the existing Phase 4 lifecycle path. Vendor-only benchmarks, synthetic counterfactuals, local proxies, and templates or dry runs were insufficient. Compiled Verilator availability and HDL design-scope changes affected prototype verification only. Public programmable-baseline updates could refresh assumptions but could not, by themselves, imply a physicalized win.

The closure report projected the campaign into seven claim rows:

Claim	Disposition
full_frontier_fixed_weight_physicalization	rejected under current evidence
safety_filter_performance_superiority	falsified under stronger programmable baseline
hybrid_architecture_failure_mode_value	retained as architecture and failure-mode study
prototype_hdl_evidence	retained as bounded prototype evidence
future_measured_reopen_path	complete but inactive absent actual measured evidence
non_safety_target_robustness	no calibrated current superiority claim
campaign_deferral_state	closed under current evidence, deferred until a valid measured package

The closure summary preserved the same endpoint values: `current_superiority_claim_count = 0`, `actual_reopen_candidate_count = 0`, `new_reopen_gate_count = 0`, `current_artifacts_reopen = false`, `phase2_hybrid_workload_wins = 0`, and `robust_calibrated_physicalized_win_count = 0`.

The archive index made that endpoint machine-checkable. It indexed 54 canonical artifacts with existence, byte size, SHA-256 hash, milestone owner, artifact class, and regeneration

command where available. The archive summary reported zero missing canonical artifacts and zero zero-size canonical artifacts. It also preserved the same endpoint counters and documented two known noncanonical warnings: generated intermediate reports and root prompt/log files.

9. Post-Closure Checks and Public-Baseline Refresh

Post-closure checks addressed three bounded maintenance questions: whether local HDL tooling had changed, whether canonical artifacts contradicted the endpoint, and whether newer public programmable-baseline evidence should refresh the baseline prior. None reopened physicalized performance superiority.

The toolchain condition probe found Verilator, Yosys, and Graphviz available. Verilator lint, Yosys eval, Yosys synthesis, and Graphviz artifact checks passed. Compiled Verilator simulation remained blocked by the environment because make and a C++ compiler were unavailable. The probe therefore refreshed prototype evidence but did not change the prototype status. It recorded `compiled_verilator_available = false`, `compiled_verilator_status = blocked_environment`, `prototype_claim_reopened = false`, `performance_claim_reopened = false`, and the endpoint counters at zero or false.

The campaign invariant checker reviewed 17 artifacts: eight JSON summaries and nine Markdown reports. It checked fields such as `current_superiority_claim_count`, `actual_reopen_candidate_count`, `new_reopen_gate_count`, `current_artifacts_reopen`, and `performance_claim_reopened`. The result was zero machine-readable contradictions. Warning-level ambiguous text rows were preserved as reader-risk flags, but they did not assert a current physicalized-weight win or actual reopen.

The public programmable-baseline recency probe identified MLPerf Inference v6.0, published by MLCommons on 2026-04-01, as newer than the earlier public MLPerf reference set [10]-[13]. The official v6.0 results repository was treated as the primary machine-readable source for future baseline refresh [13]. NVIDIA’s MLPerf page was retained as secondary vendor context only [14]. The probe recommended a model refresh and reported that public sources did not reopen the physicalized claim.

The subsequent public-baseline prior refresh ingested 12 rows from 520 available primary MLCommons v6.0 result rows. The selected rows covered public throughput-like benchmark metadata for datacenter accelerator submissions, including model/scenario combinations such as `deepseek-r1` and `gpt-oss-120b`. The mapping accepted these rows only as bounded programmable-system strength context. It found zero direct energy calibration rows and zero safety-filter direct workload rows. The refresh decision was `strengthen_programmable_null`, with the Phase 2 downgrade preserved.

The public-baseline synthesis integrated this refresh into the canonical record. It reported `public_baseline_refresh_integrated = true`, `latest_mlperf_inference_release = MLPerf Inference v6.0`, `primary_mlcommons_rows_ingested = 12`, `direct_energy_calibration_rows = 0`, `safety_filter_direct_workload_rows = 0`, `programmable_null_effect = strengthened_or_preserved`, `phase2_downgrade_preserved = true`, and `phase4_reopen_condition_unchanged = true`.

The important distinction is between B and H . Public benchmark evidence can update or strengthen B , the best programmable-baseline side of the comparison. It does not supply H , the measured hybrid physicalized total under the same safety/filter workload accounting. Because the Phase 4 reopen condition requires both measured hybrid evidence and measured best programmable-baseline evidence in the same lifecycle-valid package, public MLPerf evidence alone cannot satisfy the reopen rule.

10. Trigger-Gated Endpoint and Residual Work

The final endpoint is trigger-gated at M-PUBLICBASE-SYNTH-1. Later admission-control records did not add scientific evidence, executable validation, milestone state, or research artifacts. They formalized that no further research work should be opened unless a validated trigger appears.

The endpoint state is:

Endpoint field	Current value
current_superiority_claim_count	0
actual_reopen_candidate_count	0
new_reopen_gate_count	0
current_artifacts_reopen	false

The trigger-gating check reviewed the admissible trigger classes and found all absent: no lifecycle-valid measured production, shadow, or canary hybrid evidence; no relevant compiled-HDL capability change for M-PROTO-1; no materially new primary public-data mapping scope beyond the public-baseline milestones; and no nonduplicative handoff artifact class. The controlling decision was PIVOT, meaning the work should not treat no-op admission handling or watch-state reconfirmation as campaign progress.

A final admission check recorded the same state as ADMISSION_BLOCKED_NO_CYCLE. No scientific evidence, validation runs, or research artifacts were added. The audit again issued PIVOT, preserving the trigger-gated endpoint and preventing duplicate endpoint validation from being counted as a new research result.

The final audit summary supplied to this final-report stage has `promise_check = unknown`, no milestone-status distribution, no confidence-tagged milestone state, no residual-debt items, no future-work items, no severity-count findings, and no reconciliation events. It does report complete figure ledger coverage: 33 figures present and 33 figures in the ledger, with no missing or orphan figures. Because the audit summary provides no residual-debt or future-work anchors, this report does not invent new research directions beyond the already defined trigger condition.

The remaining actionable work is therefore conditional:

Trigger	Effect
lifecycle-valid measured production/shadow/canary hybrid evidence make and a C++ compiler become available for compiled Verilator simulation	evaluate through the Phase 4 reopen condition rerun prototype verification for M-PROTO-1; this affects prototype correctness, not performance superiority by itself
materially new primary public-data mapping scope appears nonduplicative handoff artifact requirement appears	refresh programmable-baseline prior B; this does not reopen without measured hybrid H produce that handoff artifact without changing scientific claims unless new evidence supports it

Absent one of those triggers, the campaign remains closed under current evidence. Full fixed frontier-model physicalization is rejected under the present record. Safety/filter physicaliza-

tion remains a bounded architecture, verification, failure-mode, and evidence-gating study. It is not a current performance or economic winner over strong programmable baselines.

Conclusions

The investigation ends in a negative but actionable state. Full fixed frontier-model physicalization is rejected under the present evidence. The one retained candidate, a bounded safety/filter fast path, remains valuable as an architecture and evidence-scaffold study, but it is not a current performance or economic winner over strong programmable baselines.

The decisive result is the Phase 2 downgrade. After calibrated modeling and stronger-baseline replay, programmable accelerators won nine of ten workload regimes, optimized software/runtime won one, and the hybrid physicalized safety/filter path won none. The previously preserved high-volume stable moderation case favored the programmable baseline by 1,471,448,845.624272 pJ-equivalent/day. Subsequent work did not try to bypass that result. It built the measurement and lifecycle machinery required for a future valid challenge.

The reopen rule is now explicit. A future package must be measured, privacy-safe, provenance-attested production, shadow-production, or canary evidence. It must preserve hashes and schema compatibility, map to a known threshold scenario, pass trace validation and admissible ingestion, include nonzero request volume and accepted fast-path volume, measure both the hybrid total H and best programmable baseline B under identical accounting, cross the quantitative threshold, and satisfy $UCB_{\alpha}(H - B) < 0$. Current artifacts are templates, synthetic fixtures, local proxies, rehearsals, and validators; they do not meet that condition.

The endpoint is trigger-gated at M-PUBLICBASE-SYNTH-1. The current counters are zero current physicalized superiority claims, zero actual reopen candidates, zero new reopen gates, and `current_artifacts_reopen = false`. Public MLPerf Inference v6.0 evidence updates programmable-baseline context, but it does not provide measured hybrid safety/filter evidence and therefore does not reopen the claim. The final audit summary supplied no residual-debt or future-work anchors, no severity-count findings, no reconciliation events, and `promise_check=unknown`; this report therefore limits future work to the defined triggers rather than inventing additional directions.

Implementation Details

The final conclusions cite the following workspace files as the main result-producing artifacts. The list is scoped to evidence needed to reproduce or inspect the findings reported above; intermediate reports, drafts, and one-off probes are excluded.

- `physicalized-weights/scripts/breakeven_model.py` — Normalized break-even sweep and default winner counts used in Section 1.
- `physicalized-weights/scripts/symbolic_breakeven.wls` — Wolfram break-even special-case derivation used in Section 1.
- `physicalized-weights/scripts/target_scoring.py` — Target and anti-target ranking used in Section 2.
- `physicalized-weights/scripts/fallback_policy_sim.py` — Hybrid safety/filter routing and fallback simulation used in Section 2.
- `physicalized-weights/scripts/prototype_safety_filter.py` — Prototype classifier vectors, routing distribution, and baseline comparison used in Section 3.
- `physicalized-weights/scripts/verify_prototype_closure.py` — Prototype closure evidence tying Python, HDL, Yosys, Verilator lint, Graphviz, and hash freshness in Section 3.
- `physicalized-weights/scripts/build_final_synthesis.py` — Initial evidence-labeled synthesis

and evidence map summarized in Section 3.

- `physicalized-weights/hdl/safety_filter_core.sv` — Combinational safety/filter HDL core described in Section 3.
- `physicalized-weights/hdl/run_yosys_eval.py` — Yosys evaluation of the HDL core against Python golden vectors in Sections 3 and 9.
- `physicalized-weights/hdl/safety_filter_core.py` — Yosys synthesis and DOT-export flow supporting Sections 3 and 9.
- `physicalized-weights/hdl/safety_filter_core_tb.cpp` — Retained compiled-simulation test-bench whose environment blocker is discussed in Sections 3 and 10.
- `physicalized-weights/scripts/calibrated_breakeven.py` — Explicit-unit calibrated model and scenario counts used in Section 4.
- `physicalized-weights/scripts/workload_trace_generator.py` — Synthetic workload regimes and fast-path viability overlay used in Section 4.
- `physicalized-weights/scripts/stronger_baseline_model.py` — Equal-workload stronger-baseline replay and Phase 2 falsification result used in Section 4.
- `physicalized-weights/scripts/build_phase2_synthesis.py` — Phase 2 claim matrix and downgrade synthesis used in Section 4.
- `physicalized-weights/scripts/local_overhead_benchmark.py` — Local proxy timing medians and production-measurement gap evidence used in Section 5.
- `physicalized-weights/scripts/production_trace_validator.py` — Production trace schema and validator rules used in Section 5.
- `physicalized-weights/scripts/reopen_thresholds.py` — Quantitative reopen thresholds and current zero-hybrid-win accounting used in Section 6.
- `physicalized-weights/scripts/symbolic_reopen_thresholds.wls` — Wolfram reopen-threshold special cases used in Section 6.
- `physicalized-weights/scripts/trace_ingestion_path_evaluator.py` — Admissible ingestion-path classification used in Section 6.
- `physicalized-weights/scripts/reopen_pipeline_demo.py` — End-to-end validation, ingestion, provenance, and threshold pipeline used in Section 6.
- `physicalized-weights/scripts/evidence_pack_replay.py` — Replayable evidence-pack manifest harness used in Section 6.
- `physicalized-weights/scripts/build_phase3_reopen_synthesis.py` — Phase 3 reopen-pathway synthesis summarized in Section 6.
- `physicalized-weights/scripts/evidence_acquisition_readiness.py` — Pre-collection readiness classification used in Section 6.
- `physicalized-weights/scripts/evidence_pack_template_dryrun.py` — Operator evidence-pack dry-run templates and blocking checks used in Section 6.
- `physicalized-weights/scripts/evidence_pack_intake_rehearsal.py` — Synthetic-safe intake handoff rehearsal used in Section 6.
- `physicalized-weights/scripts/reopen_uncertainty_protocol.py` — Uncertainty-aware UCB reopen rule used in Section 6.
- `physicalized-weights/scripts/evidence_package_lifecycle.py` — Lifecycle state machine and terminal-state accounting used in Section 7.
- `physicalized-weights/scripts/build_phase4_reopen_synthesis.py` — Canonical Phase 4 reopen conjunction and claim matrix used in Section 7.
- `physicalized-weights/scripts/target_robustness_stress.py` — Target-class robustness stress test used in Section 8.
- `physicalized-weights/scripts/build_campaign_deferral_watchlist.py` — Deferral watchlist and trigger governance used in Sections 8 and 10.
- `physicalized-weights/scripts/build_campaign_closure_report.py` — Closure claim dispositions and endpoint counters used in Section 8.

- `physicalized-weights/scripts/build_closure_archive_index.py` — Canonical archive index and artifact coverage evidence used in Section 8.
- `physicalized-weights/scripts/toolchain_condition_probe.py` — Post-closure toolchain condition probe used in Section 9.
- `physicalized-weights/scripts/campaign_invariant_checker.py` — Cross-artifact invariant checker used in Section 9.
- `physicalized-weights/scripts/public_baseline_recency_probe.py` — MLPerf recency and source materiality screen used in Section 9.
- `physicalized-weights/scripts/public_baseline_prior_refresh.py` — MLCommons v6.0 public-baseline prior refresh used in Section 9.
- `physicalized-weights/scripts/build_public_baseline_synthesis.py` — Public-baseline synthesis addendum and endpoint preservation used in Section 9.

References

- [1] RISC-V International, “RISC-V FAQ,” RISC-V International. <https://riscv.org/about/faq/>
- [2] RISC-V International, “RISC-V Technical Specifications,” RISC-V International. <https://docs.riscv.org/reference/home/index.html>
- [3] The gem5 Project, “About gem5,” gem5. <https://www.gem5.org/about/>
- [4] OpenROAD Project, “OpenROAD,” OpenROAD. <https://openroad.ergodex.ai/>
- [5] LLVM CIRCT Project, “CIRCT,” GitHub. <https://github.com/llvm/circt>
- [6] IBM Research, “How can analog in-memory computing power transformer models?,” IBM Research. <https://research.ibm.com/blog/how-can-analog-in-memory-computing-power-transformer-models>
- [7] Mark Horowitz, “1.1 Computing’s Energy Problem (and What We Can Do about It),” 2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC), 2014. <https://doi.org/10.1109/ISSCC.2014.6757323>
- [8] Mark Horowitz, “Computing’s Energy Problem (and What We Can Do about It),” slide transcript/mirror of ISSCC 2014 energy table. <https://doczz.net/doc/9135487/computing-s-energy-problem--and-what-we-can-do-about-it->
- [9] NVIDIA, “NVIDIA H100 Tensor Core GPU,” product specification page. <https://www.nvidia.com/en-us/data-center/h100/>
- [10] MLCommons, “MLPerf Inference: Datacenter benchmark documentation,” MLCommons. <https://docs.mlcommons.org/inference/>
- [11] MLCommons, “MLPerf Inference v6.0 Results,” MLCommons, 2026. <https://mlcommons.org/2026/04/mlperf-inference-v6-0-results/>
- [12] MLCommons, “MLPerf Inference v5.1 Results,” MLCommons, 2025. <https://mlcommons.org/2025/09/mlperf-inference-v5-1-results/>
- [13] MLCommons, “MLPerf Inference Results v6.0,” GitHub, 2026. https://github.com/mlcommons/inference_results_v6.0
- [14] NVIDIA, “MLPerf AI Benchmarks,” NVIDIA. <https://www.nvidia.com/en-us/data-center/resources/mlperf-benchmarks/>